Mobil Uygulama Geliştirme - Sınav Çalışma Dosyaları

Ders deposu: https://github.com/zeki-ozen/MobileAppDevelopment

Bu paket, sınavda hızlı konu/kod araması yapabilmen için haftalara ayrılmış Türkçe özetler içerir. Her PDF'de **"30 saniyelik hızlı bakış"**, açıklamalı örnekler, kritik karşılaştırmalar, sık hatalar ve mini soru-cevap bulunur.

Dosya	İçerik	Kaynak durumu
Hafta_01.pdf	Mobil uygulama geliştirmeye giriş	W1 yok: tamamlayıcı
Hafta_02.pdf	JavaScript, ES6, diziler, modüller	W2 kaynaklı
Hafta_03.pdf	React, JSX, props/state, hooks, formlar	W3 kaynaklı
Hafta_04.pdf	React Native, Expo, state, hooks	W4 kaynaklı
Hafta_05.pdf	Bileşik RN görev uygulaması ve hata desenleri	W5 yok: tamamlayıcı
Hafta_06.pdf	View/Text/Image/TextInput/FlatList vb.	W6 kaynaklı
Hafta_07.pdf	AsyncStorage, SQLite, Realm, MMKV, MySQL	W7 kaynaklı
Hafta_08.pdf	Fetch/Axios, REST, async/await, hata/loading	W8 kaynaklı
Hafta_09.pdf	Expo kamera/bildirim, haritalar, izinler	W9 kaynaklı

Sınavda nasıl kullanılır?

1. **PDF içinde ara (Ctrl+F / Cmd+F):** `useEffect`, `FlatList`, `fetch`, `AsyncStorage`, `CameraView`, `Marker` gibi ifadeleri ara.
2. **Kod kullanmadan önce:** Üstteki `import` satırlarını, kurulu paketi, çalıştırma ortamını ve state tanımlarını kontrol et.
3. **Sürüm farkları:** Expo ve React Native API'leri güncellenebilir. Depodaki eski bir API'nin güncel SDK'da farklı olabileceği yerler özellikle not edildi.
4. **Kaynak niteliği:** Bunlar ders materyalinden türetilmiş çalışma notlarıdır; öğretim elemanının verdiği tek kaynak veya sınav sorularının garantili kapsamı değildir.

Sınav içi problem → sayfa seçimi

Sorun	Açılacak hafta
Diziyi dönüştür / veri tipini düzelt	2
React props/state/hooks mantığını anlat	3–4
Form + liste + silme/güncelleme	5–6
Kullanıcı tercihini kalıcı tut	7
API çağır / HTTP hata yönet	8
Kamera, bildirim veya harita ekle	9

Hazırlanış: 2 Kasım 2026. Kaynak: GitHub deposunun `main` dalındaki erişilebilir dosya adları ve ders notları. Kod örnekleri eğitim amacıyla sadeleştirilmiştir. Bazı örnekler sürüm ve platforma göre ek kurulum gerektirir.

1. HAFTA | Mobil Uygulama Geliştirmeye Giriş

> **Kaynak durumu:** Depoda `W1/` klasörü yoktur. Bu belge, `W4/1-react-native-intro.md` dosyasındaki "Week 1: Introduction to Mobile App Development" ön koşuluna dayanarak hazırlanmış **tamamlayıcı giriş ve tekrar notudur**; öğretim elemanının 1. hafta ders içeriğinin birebir karşılığı olduğu iddia edilmez.

30 saniyelik hızlı bakış

- **Native:** Android için Kotlin/Java, iOS için Swift/Objective-C. Platforma özgü API ve araçlar.
- **Cross-platform:** React Native gibi tek JavaScript/TypeScript kod tabanıyla Android ve iOS arayüzü geliştirme.
- **Web / PWA:** Tarayıcı tabanlı; platform entegrasyonu ve dağıtımı farklıdır.
- **React:** Web arayüzü için bileşen tabanlı kütüphane. **React Native:** Gerçek mobil bileşenler (`View`, `Text`, `Image`) kullanır.
- **Expo:** React Native geliştirmeyi, başlatmayı ve cihazda denemeyi kolaylaştıran araç/SDK ekosistemi.

1. Kavram haritası

Kavram	Kısaca	Sınavda ayırt et
Android / iOS	Mobil işletim sistemi	Platformlar arası farklar
Native app	Platformun yerel teknolojisiyle yazılan uygulama	Platform API erişimi
Cross-platform app	Ortak kod tabanını birden çok platformda kullanma	Kod paylaşımı, platform uyarlaması
React	Bileşen + props + state + JSX	Web tarafında HTML/DOM
React Native	React yaklaşımının mobil uygulaması	HTML yerine `View`, `Text`
Expo Go	Geliştirme sırasında uygulama deneme aracı	Her yerel modül Expo Go'da çalışmayabilir
Metro	React Native JavaScript paketleyicisi	Derleme/çalıştırma altyapısı
Emulator / Simulator	Sanal Android / iOS cihazı	Gerçek donanım davranışı farklı olabilir

2. Uygulama geliştirme akışı

1. **Gereksinimi belirle:** Hedef kullanıcı, çözülecek problem, ekranlar ve veri gereksinimi.
2. **Arayüzü tasarla:** Ekran hiyerarşisi, giriş alanları, erişilebilirlik, tutarlı boşluklar.
3. **Proje oluştur:** Node.js, paket yöneticisi, VS Code ve Expo ortamını hazırla.
4. **Bileşenlerle geliştir:** Ekranı küçük `View`, `Text` ve özel bileşenlere böl.
5. **Veri akışı kur:** `props` ile aktar, `useState` ile yerel durumu sakla, API veya kalıcı depolama ekle.
6. **Test et:** Farklı ekran boyutları, kullanıcı izinleri, çevrimdışı durum, hatalar.
7. **Dağıt:** Geliştirme sürümü ile mağaza yayını birbirinden farklı süreçlerdir.

3. Kurulum ve ilk uygulama

Aşağıdaki komutlar Expo için tipik başlangıçtır. Şablonlara göre giriş dosyasının adı `App.js`, `App.tsx` veya `app/` yönlendirme yapısı olabilir.

```
npx create-expo-app@latest ilk-uygulamam
cd ilk-uygulamam
npx expo start
```

En basit React Native bileşeni:

```
import { View, Text, StyleSheet } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Text>Merhaba mobil dünya!</Text>
    </View>
  );
}
const styles = StyleSheet.create({
  container: { flex: 1, alignItems: 'center', justifyContent: 'center' },
});
```

Satır satır: `import` bileşenleri alır; `App` fonksiyon bileşenidir; `return` JSX döndürür; `View` kapsayıcıdır; `Text` yazıyı gösterir; `StyleSheet.create` stil nesnelerini düzenler; `flex:1` kullanılabilir alanı kaplar.

4. Web React ile React Native'i karıştırma

Web React	React Native
`<div>`	`<View>`
`<p>Merhaba</p>`	`<Text>Merhaba</Text>`
`<img src="..." />`	`<Image source={{uri:'...'}} />`
`onClick`	`onPress` (örn. Button/Pressable)
CSS dosyası / className	`style` nesnesi / `StyleSheet`
React Router	Genelde React Navigation veya Expo Router

5. Sınav için kısa cevaplar

S: React Native'in React'ten farkı nedir? React web arayüzünde DOM/HTML; React Native mobil platformun yerel görsel bileşenleriyle çalışır. Her ikisi de bileşen/props/state mantığını paylaşır.

S: `props` ve `state` farkı? `props` üst bileşenden iletilen salt-okunur girdidir; `state` bileşenin güncellenebilir verisidir ve değiştiğinde yeniden render tetiklenir.

S: Expo neden kullanılır? Başlangıç, paket/SDK entegrasyonu, geliştirme sunucusu ve cihazda önizlemeyi kolaylaştırır. Yerel modül desteğini proje türüne göre kontrol etmek gerekir.

S: UI neden bileşenlere ayrılır? Tekrar kullanılabilirlik, test, bakım ve sorumlulukları ayırma için.

6. Sınav öncesi kontrol

- Native / cross-platform / web ayrımını birer cümleyle anlatabiliyorum.
- `View`, `Text`, `StyleSheet`, `App` fonksiyonunu tanıyabiliyorum.
- `props`, `state`, JSX terimlerini karıştırmıyorum.
- `npx expo start` neyi başlatır biliyorum.

Kaynak: https://github.com/zeki-ozen/MobileAppDevelopment/blob/main/W4/1-react-native-intro.md

2. HAFTA | JavaScript ve ES6+ Temelleri

> **Kaynak:** `W2/README.md` ve `W2/topics/00–10` başlıkları. Bu dosya repo içeriğinin sınav odaklı Türkçe özetidir.

30 saniyelik hızlı bakış

`const` varsayılan seçim; yeniden atama gerekiyorsa `let`. `===` tip dönüştürmeden karşılaştırır. `??` yalnızca `null` / `undefined` için varsayılan değer verir; `||` tüm falsy değerleri ele alır. Dizi dönüştürme `map`, eleme `filter`, tek değere indirgeme `reduce`. Modül paylaşımı `export` / `import`.

1. Değişkenler: var, let, const

Anahtar kelime	Scope	Yeniden atama	Yeniden tanımlama
`var`	Fonksiyon	Evet	Aynı scope'ta evet
`let`	Blok `{}`	Evet	Aynı blokta hayır
`const`	Blok `{}`	Hayır	Aynı blokta hayır

```
const ad = 'Ayşe';
let puan = 70;
puan += 10;
const ogrenci = { ad: 'Ayşe' };
ogrenci.ad = 'Ece';  // geçerli: nesnenin içeriği değişiyor
// ogrenci = {};    // hata: const değişkenine tekrar atama
```

Tuzak: `const` nesneyi derinlemesine değişmez yapmaz. `let` / `const` tanımlanmadan önce erişim temporal dead zone nedeniyle hata verir; `var` eski kodda istenmeyen hoisting davranışı doğurabilir.

2. Veri tipleri ve karşılaştırma

Primitive: `string`, `number`, `boolean`, `undefined`, `null`, `bigint`, `symbol`. **Reference:** object, array, function. `typeof null` sonucu tarihsel bir JavaScript özelliği olarak `'object'` tir. `Array.isArray([])` diziyi doğru ayırt eder.

```
console.log(typeof 42);         // 'number'
console.log(typeof null);       // 'object'
console.log(Array.isArray([]));  // true
console.log(5 == '5');          // true (tip dönüşümü)
console.log(5 === '5');         // false
console.log(null ?? 'yok');     // 'yok'
console.log(0 ?? 10);           // 0
console.log(0 || 10);           // 10
```

Falsy değerler: `false`, `0`, `-0`, `0n`, `''`, `null`, `undefined`, `NaN`. Boş dizi `[]` ve boş nesne `{}` truthy'dir.

3. Operatörler, koşullar, döngüler

```
const not = 84;
const harf = not >= 90 ? 'AA' : not >= 70 ? 'BB' : 'FF';
if (not >= 50 && not <= 100) console.log('Geçti');

const dersler = ['JS', 'React', 'RN'];
for (const ders of dersler) console.log(ders); // değerler
for (const index in dersler) console.log(index); // anahtar/index
```

`switch` çoklu sabit durum için uygundur; `break` olmazsa devam ederek sonraki case'e de girebilir. `forEach` yan etki/ekrana yazma için; `map` yeni dizi üretmek için. `forEach` içinde `break` kullanılamaz.

4. Fonksiyon ve arrow function

```
function topla(a, b = 0) { return a + b; }
const kare = x => x * x;
const isimYaz = ({ ad, soyad }) => `${ad} ${soyad}`;
console.log(topla(3, 4)); // 7
```

Arrow function sözdizimi kısadır ve kendi `this` bağlamını oluşturmaz. Tek ifade için otomatik dönüş vardır; süslü parantez kullanırsan açık `return` yazmalısın.

```
const ikiKat = sayilar => sayilar.map(x => x * 2); // doğru
const hatali = x => { x * 2; }; // undefined, return yok
const dogru = x => { return x * 2; };
```

5. Dizi yöntemleri: sınavın ana tablosu

Yöntem	Girdi -> Çıktı	Kullanım
`map(fn)`	Dizi -> yeni dizi	Her elemanı dönüştür
`filter(fn)`	Dizi -> yeni dizi	Koşulu sağlayanları seç
`find(fn)`	Dizi -> eleman/undefined	İlk eşleşeni bul
`some(fn)`	Dizi -> boolean	En az biri uygun mu?
`every(fn)`	Dizi -> boolean	Hepsi uygun mu?
`reduce(fn, init)`	Dizi -> tek sonuç	Topla, grupla, birleştir
`forEach(fn)`	Dizi -> undefined	Yan etki gerçekleştir

```
const sayilar = [1, 2, 3, 4, 5];
const ciftler = sayilar.filter(x => x % 2 === 0); // [2,4]
const kareler = sayilar.map(x => x ** 2);          // [1,4,9,16,25]
const toplam = sayilar.reduce((t, x) => t + x, 0);// 15
const sonuc = sayilar.find(x => x > 3);           // 4
```

6. Spread, rest, destructuring, optional chaining

```
const eski = { ad: 'Can', not: 70 };
const yeni = { ...eski, not: 90 }; // değişmez güncelleme
const [ilk, ...kalan] = [5, 7, 9]; // ilk=5, kalan=[7,9]
const { ad, not: dersNotu } = yeni;
const sehir = kullanici?.adres?.sehir ?? 'Bilinmiyor';
```

Dikkat: Spread iç içe nesnelerde **shallow copy** üretir; iç nesneler aynı referansı paylaşabilir. `?.` olmayan yola güvenli erişim sağlar.

7. Asenkronluk ve modüller

```
// math_utils.js
export const ikiIleCarp = n => n * 2;
export default function selamla(ad) { return `Merhaba ${ad}`; }

// main.js
import selamla, { ikiIleCarp } from './math_utils.js';
console.log(selamla('Ece'), ikiIleCarp(4));
```

`export default` bir modülde en çok bir adet; named export birden çok olabilir. Tarayıcıda ES modülü için `<script type="module">` ve genelde yerel sunucu kullan. Node.js çalıştırmada modül ayarları fark yaratabilir. `Promise` ve `async/await` ayrıntıları 8. haftada var.

8. Muhtemel soru ve cevapları

1. **`let` ile `const`?** İkisi blok kapsamlıdır; `const` yeniden atamaya izin vermez.
2. **`==` ile `===`?** `==` tip dönüşümü yapabilir, `===` tip ve değeri kıyaslar.
3. **`map` ile `forEach`?** `map` dönüşümden yeni dizi üretir; `forEach` yeni dizi döndürmez.
4. **`null` ile `undefined`?** `null` bilerek boş değer; `undefined` atanmadı/yok.
5. **`??` ile `||`?** `??` yalnızca nullish; `||` falsy için fallback verir.

9. Sınavda hızlı hata ayıklama

- `undefined` çıktı: Arrow function'da `return` var mı? Yanlış property mi?
- Liste boş: `filter` koşulu mu yanlış? Sayı/string karşılaştırması mı?
- `Cannot read properties...`: `?.` / veri kontrolü eksik olabilir.
- State güncellemesinde veri kayboldu: Nesne spread kullanılmış mı?

Kaynak: https://github.com/zeki-ozen/MobileAppDevelopment/tree/main/W2

3. HAFTA | React Temelleri ve Hooks

> **Kaynak:** `W3/README.md` , ders notları (`00` – `14`) ve eşlik eden `.jsx` örnekleri. Repo bu klasörde hem temel hem ileri düzey React konularına yer verir.

30 saniyelik hızlı bakış

Component fonksiyon olarak JSX döndürür. **Props** dışarıdan veri (salt-okunur), **state** içeride güncellenen veridir. `useState` güncellemeleri ekranı yeniler; `useEffect` yan etki/abonelik için; liste JSX'inde **kararlı** `key` gerekir. Form alanında `value` + `onChange` kontrollü bileşen desenidir.

1. Component + JSX + Props

```
function Selamlama({ ad = 'Misafir' }) {
  return <h2>Merhaba, {ad}!</h2>;
}
export default function App() {
  return <main><Selamlama ad="Ece" /><Selamlama /></main>;
}
```

- React bileşen adı **büyük harfle** başlar (`Selamlama`).
- JSX içinde JS ifadesi `{...}` ile değerlendirilir.
- JSX tek kök eleman veya `<>...</>` fragment döndürmelidir.
- `className` web React'te kullanılır; React Native'de `style` kullanılır.
- `props` bileşenin içinden değiştirilmez; değişiklik gerekiyorsa state yönetilir.

2. State ve immutability

```
import { useState } from 'react';
export default function Sayac() {
  const [sayi, setSayi] = useState(0);
  return (
    <section>
      <p>Değer: {sayi}</p>
      <button onClick={() => setSayi(prev => prev + 1)}>Artır</button>
    </section>
  );
}
```

Kural: Önceki duruma dayanıyorsan `setSayi(prev => prev + 1)` kullan. State'i `sayi++` ile doğrudan değiştirme. Nesne/dizide spread veya `map` / `filter` ile **yeni referans** oluştur.

```
setKullanici(prev => ({ ...prev, ad: 'Yeni' }));
setUrunler(prev => prev.filter(u => u.id !== silinecekId));
setUrunler(prev => prev.map(u => u.id === hedefId ? {...u, tamam: true} : u));
```

3. Event, koşullu render, liste

```
const urunler = [{ id: 'a', ad: 'Kalem' }, { id: 'b', ad: 'Defter' }];
function Liste({ girisYapildi }) {
  if (!girisYapildi) return <p>Lütfen giriş yapın.</p>;
  return <ul>{urunler.map(u => <li key={u.id}>{u.ad}</li>)}</ul>;
}
```

`onClick={islem}` fonksiyonun kendisini verir; `onClick={islem()}` render sırasında **hemen çağırır** (çoğu durumda hata). React Native karşılığı çoğunlukla `onPress` olur. `key` olarak mümkünse veri kimliği (`id`) kullan; dizi indeksini, liste sırası değişiyorsa tercih etme.

4. useEffect: ne zaman çalışır?

```
import { useEffect, useState } from 'react';
function Saat() {
  const [zaman, setZaman] = useState(new Date());
  useEffect(() => {
    const id = setInterval(() => setZaman(new Date()), 1000);
    return () => clearInterval(id); // cleanup
  }, []);
  return <p>{zaman.toLocaleTimeString()}</p>;
}
```

Kod	Anlam
`useEffect(fn)`	Her commit sonrası effect (gerekliyse kaçın)
`useEffect(fn, [])`	Mount sonrası; cleanup unmount'ta
`useEffect(fn, [id])`	`id` değiştiğinde tekrar
`return () => ...`	Timer/abonelik/istek temizliği

Önemli: React Strict Mode geliştirme modunda effect kurulum/temizliğini ek kez çalıştırabilir; geliştirme gözlemiyle üretim davranışını karıştırma.

5. Hook karşılaştırma tablosu

Hook	Ne için?	Sık hata
`useState`	Bileşen verisi	Doğrudan mutasyon
`useEffect`	API, timer, abonelik	Eksik dependency / cleanup
`useMemo`	Hesaplanan değeri önbellekle	Gereksiz kullanım
`useCallback`	Fonksiyon referansını önbellekle	Yanlış dependency
`useRef`	Render dışı mutable referans, DOM	Güncellemesini ekranda beklemek
`useContext`	Üst sağlayıcıdan ortak veri	Provider olmaması

Hook kuralı: Hook'ları fonksiyon bileşenin veya custom hook'un **en üst seviyesinde** çağır; döngü, koşul veya normal fonksiyon içinde çağırma.

6. Form: controlled input

```
import { useState } from 'react';
function Form() {
  const [eposta, setEposta] = useState('');
  const [mesaj, setMesaj] = useState('');
  const gecerli = eposta.includes('@');
  function gonder(e) {
    e.preventDefault();
    setMesaj(gecerli ? 'Kaydedildi' : 'Geçersiz e-posta');
  }
  return <form onSubmit={gonder}>
    <input value={eposta} onChange={e => setEposta(e.target.value)} />
    <button type="submit">Gönder</button><p>{mesaj}</p>
  </form>;
}
```

Veri paylaşma: İki kardeş bileşen aynı veriye ihtiyaç duyuyorsa state'i en yakın ortak üst bileşene taşı (**lifting state up**). Daha geniş kapsam için Context API kullanılabilir; gereksiz global durum kurma.

7. Stil, routing ve web / mobil farkı

- React web: `className`, CSS / CSS Modules / inline style; sayfalar için React Router.
- React Native: `StyleSheet`, `View`, `Text`; gezinme için React Navigation veya Expo Router. **React Router kodunu RN uygulamasına doğrudan kopyalama.**
- `useEffect` veri alırken loading/error durumlarını ayrıca yönetmek gerekir (8. hafta).

8. Sınav tipinde mini soru-cevap

S: Props mu state mi? Üstten gelen ve bileşenin değiştirmemesi gereken veriler props; etkileşimle değişecek veriler state.

S: `key` neden önemli? Liste elemanlarının kimliğini koruyarak güncellemelerin doğru eşleşmesine yardım eder.

S: `useEffect` içinde neden cleanup? Timer/abonelik gibi kaynakları temizleyerek sızıntı ve yinelenen etkiyi önlemek için.

S: `setState` neden fonksiyonel biçimde? Önceki state'e bağlı ardışık güncellemelerde güncel değeri kullanmak için.

Kaynak: https://github.com/zeki-ozen/MobileAppDevelopment/tree/main/W3

4. HAFTA | React Native, Expo, Props, State ve Hooks

> **Kaynak:** `W4/1-react-native-intro.md – W4/6-react-native-hooks.md` . Repo konuları mobil bağlama uyarlayan ayrı örneklerle anlatır.

30 saniyelik hızlı bakış

React Native = React bileşen modeli + mobil UI bileşenleri. `View` kapsayıcı, `Text` metin, `StyleSheet` stil; tıklama `onPress`. **Props** üstten alt bileşene, **state** değişen değer, **effect** yan etki. Expo hızlı başlangıç ve cihazda test içindir.

1. Expo ve proje akışı

```
npx create-expo-app@latest RNOrnek
cd RNOrnek
npx expo start
```

Repo örnekleri sıklıkla `App.js` dosyası üzerinden açıklanır; güncel Expo şablonlarında `app/` dizini ve Expo Router bulunabilir. Uygulamada kullandığın şablona göre giriş dosyasını seç.

React (web)	React Native (mobil)
`<div>`	`<View>`
`<p>` / `<span>`	`<Text>`
`<button onClick>`	`<Button onPress>` / `<Pressable>`
CSS	`style={{...}}` / `StyleSheet.create`
DOM	Native UI bileşenleri

2. İşleyen örnek: props + state + event

```
import { useState } from 'react';
import { View, Text, Button, StyleSheet } from 'react-native';

function SkorKart({ baslik, skor }) {
  return <Text style={styles.title}>{baslik}: {skor}</Text>;
}
export default function App() {
  const [puan, setPuan] = useState(0);
  return (
    <View style={styles.container}>
      <SkorKart baslik="Puan" skor={puan} />
      <Button title="+1" onPress={() => setPuan(p => p + 1)} />
      <Button title="Sıfırla" onPress={() => setPuan(0)} />
    </View>
  );
}
const styles = StyleSheet.create({
  container: { flex: 1, padding: 24, justifyContent: 'center' },
  title: { fontSize: 24, marginBottom: 12 },
});
```

Akış: `App` state'i tutar → `puan` değişir → `SkorKart` yeni `skor` props'unu alır → ekranda güncel sayı görünür. `SkorKart` içinde `skor++` yapmak doğru değildir.

3. useState + TextInput formu

```
import { useState } from 'react';
import { View, Text, TextInput, Button } from 'react-native';
export default function FormEkrani() {
  const [ad, setAd] = useState('');
  const [kayit, setKayit] = useState('');
  return <View style={{ padding: 20, gap: 10 }}>
    <TextInput placeholder="Adınız" value={ad} onChangeText={setAd}
               style={{ borderWidth: 1, padding: 10 }} />
    <Button title="Kaydet" onPress={() => setKayit(ad.trim())} />
    <Text>Kayıt: {kayit}</Text>
  </View>;
}
```

`onChangeText`, RN'de doğrudan **string** gönderir. Web React'teki `e.target.value` burada tipik desen değildir. `TextInput value` kontrollü alanı oluşturur.

4. useEffect ve abonelik temizliği

```
import { useEffect, useState } from 'react';
import { Text } from 'react-native';
export default function Saniye() {
  const [sure, setSure] = useState(0);
  useEffect(() => {
    const id = setInterval(() => setSure(s => s + 1), 1000);
    return () => clearInterval(id);
  }, []);
  return <Text>Geçen saniye: {sure}</Text>;
}
```

- `useState(0)`: İlk değer 0; setter değişikliği render tetikler.
- `useEffect(..., [])`: Ekrana bağlandığında effect; ayrılınca cleanup.
- `useEffect(..., [x])`: `x` değişince yeniden çalışır.
- Ağ isteği, timer ve event dinleyicisi gibi yan etkiler render gövdesinde başlatılmamalıdır.

5. Stil ve mobil yerleşim

Özellik	Etki
`flex: 1`	Kullanılabilir alanı doldur
`flexDirection: 'column'`	Dikey eksen (RN varsayılanı)
`justifyContent`	Ana eksende hizala
`alignItems`	Çapraz eksende hizala
`padding`, `margin`	İç / dış boşluk
`backgroundColor`	Arka plan rengi
`fontSize`, `fontWeight`	Metin stili

Tuzak: `justifyContent` ve `alignItems` hangi eksende etkili, `flexDirection` değerine bağlıdır. `View` içine düz string koymak yerine `Text` kullan.

6. Platform farkları

```
import { Platform } from 'react-native';
const bilgi = Platform.OS === 'ios' ? 'iOS' : 'Android veya diğer';
const pad = Platform.select({ ios: 20, android: 12, default: 16 });
```

Mobil platform izinleri, ekran güvenli alanları ve native paket desteği platforma göre değişir. **Expo Go** ile özel geliştirme yapısını (development build) birbirine eşitleme.

7. Sınav örnekleri

S: TextInput'a yazınca ekran nasıl değişir? `onChangeText` → setter → state güncellenir → component yeniden render olur.

S: Props neden salt okunur? Tek yönlü veri akışını korur; güncelleme state'i tutan üst bileşende yapılır.

S: useEffect cleanup ne işe yarar? Önceki timer/aboneliği sonlandırır; gereksiz/tekrarlanan işlemi önler.

S: CSS dosyasındaki `display: grid` doğrudan kullanılır mı? React Native stilleri web CSS'in birebir tamamı değildir; `StyleSheet` ve Flexbox tabanlı yerleşim kullanılır.

Kaynak: https://github.com/zeki-ozen/MobileAppDevelopment/tree/main/W4

5. HAFTA | React Native Uygulama Alıştırmaları (Tamamlayıcı)

> **Kaynak durumu:** Depoda `W5/` bulunmuyor. Bu belge **resmî 5. hafta ders notu değildir**. `W4` (state/props/hooks) ile `W6` (temel bileşenler) arasında pratik yapmanı sağlayacak sınav odaklı tekrar materyalidir.

30 saniyelik hızlı bakış

Bir ekranın tipik akışı: **kullanıcı girişi** (`TextInput`) → **state** (`useState`) → **olay** (`onPress`) → **dizi güncellemesi** (`map` , `filter` , spread) → **liste render** (`FlatList`) → **benzersiz key**. React Native'de veri akışı ve arayüz elemanları birlikte düşünülür.

1. Tam uygulama: görev ekle, bitir, sil

Aşağıdaki örnek Expo'nun `App.js` girişine uyarlanabilir. `FlatList` eleman kimliğini `id` ile takip eder; işlemler state'i **mutasyona uğratmadan** günceller.

```
import { useState } from 'react';
import { View, Text, TextInput, Button, FlatList, Pressable } from 'react-native';

export default function App() {
  const [metin, setMetin] = useState('');
  const [gorevler, setGorevler] = useState([]);

  function ekle() {
    const baslik = metin.trim();
    if (!baslik) return;
    const yeni = { id: String(Date.now()) + Math.random(), baslik, bitti: false };
    setGorevler(eski => [yeni, ...eski]);
    setMetin('');
  }
  function tersCevir(id) {
    setGorevler(eski => eski.map(g =>
      g.id === id ? { ...g, bitti: !g.bitti } : g));
  }
  function sil(id) {
    setGorevler(eski => eski.filter(g => g.id !== id));
  }

  return <View style={{ flex: 1, padding: 20, paddingTop: 60 }}>
    <Text style={{ fontSize: 24, marginBottom: 12 }}>Görevler</Text>
    <TextInput value={metin} onChangeText={setMetin}
      placeholder="Yeni görev" style={{ borderWidth: 1, padding: 10 }} />
    <Button title="Ekle" onPress={ekle} />
    <FlatList data={gorevler} keyExtractor={item => item.id}
      ListEmptyComponent={<Text>Henüz görev yok.</Text>}
      renderItem={({ item }) => <View style={{ flexDirection: 'row', gap: 12, padding: 10 }}>
        <Pressable onPress={() => tersCevir(item.id)} style={{ flex: 1 }}>
          <Text style={{ textDecorationLine: item.bitti ? 'line-through' : 'none' }}>
            {item.baslik}
          </Text>
        </Pressable>
        <Button title="Sil" onPress={() => sil(item.id)} />
      </View>} />
  </View>;
}
```

Not: `Date.now()` + rastgele ek, bu derslik tek istemcili demo için kimlik üretir. Üretim ve senkronizasyon senaryolarında güvenilir UUID / sunucu kimliği kullan.

2. Örneği analiz et: sınavda hangi satır ne yapıyor?

Kod kalıbı	Görevi
`const [x, setX] = useState(...)`	Yerel state yaratır
`metin.trim()`	Boşlukları keser
`if (!baslik) return`	Boş kayıt eklemeyi önler
`[yeni, ...eski]`	Yeni eleman içeren yeni dizi
`map(... ? {...g, bitti: ...} : g)`	Seçili görevde immutability
`filter(g => g.id !== id)`	Eleman silme
`keyExtractor={item => item.id}`	Kararlı liste kimliği

Kod kalıbı	Görevi
`onPress={() => sil(item.id)}`	Argümanla olay tetikleme

3. Ek alıştırma: arama / filtreleme

```
const [aranan, setAranan] = useState('');
const gorunenler = gorevler.filter(g =>
  g.baslik.toLocaleLowerCase('tr-TR').includes(aranan.toLocaleLowerCase('tr-TR'))
);
// FlatList data={gorunenler}
```

Ayırım: Filtre sonucunu çoğu zaman ayrı state'e koymana gerek yok; mevcut state'ten hesaplanabilir. Büyük listelerde hesap maliyeti varsa `useMemo` değerlendirilir.

4. Yaygın hatalar ve düzeltmeleri

Hata 1 – doğrudan state mutasyonu:

```
// Yanlış:
gorevler.push(yeni);
setGorevler(gorevler);
// Doğru:
setGorevler(eski => [...eski, yeni]);
```

Hata 2 – render sırasında fonksiyon çağırma:

```
// Yanlış: onPress={sil(item.id)}
// Doğru:  onPress={() => sil(item.id)}
```

Hata 3 – listede key unutma:

```
<FlatList data={veriler} renderItem={({item}) => <Text>{item.ad}</Text>}
          keyExtractor={item => String(item.id)} />
```

Hata 4 – TextInput / web karışıklığı: React Native'de `onChangeText={setMetin}`; web React'te `onChange={e => setMetin(e.target.value)}`.

5. Sınav cevap şablonları

“Kullanıcı girdisini listeye ekle” → `TextInput` + `useState` + `onPress` + `setListe(prev => [...prev, yeni])`.

“Seçileni sil” → `setListe(prev => prev.filter(x => x.id !== hedef))`.

“Öğe durumunu değiştir” → `map` + şart + `{...x, alan: yeniDeger}`.

“Koşullu boş ekran göster” → `ListEmptyComponent` veya `liste.length === 0 ? ... : ...`.

Kaynak (köprü): https://github.com/zeki-ozen/MobileAppDevelopment/tree/main/W4 ve https://github.com/zeki-ozen/MobileAppDevelopment/tree/main/W6

6. HAFTA | React Native Temel UI Bileşenleri

> **Kaynak:** `W6/README.md` ve `W6/components/*.md`. Depoda `StudentExercise.md` dosyası README içinde anılmış olsa da taranan dalda bulunmadı; burada yalnız mevcut bileşen notlarına dayandık.

30 saniyelik hızlı bakış

`View` = kapsayıcı; `Text` = yazı; `Image` = görsel; `TextInput` = giriş; `Button` / `TouchableOpacity` / `Pressable` = etkileşim; `ScrollView` = az sayıda statik kaydırılabilir içerik; `FlatList` = büyük/değişken listeler; `SafeAreaView` = güvenli alan kavramı. Picker, RadioButton ve Checkbox birçok projede **ayrı paket** gerektirir.

1. Bileşen seçme tablosu

Bileşen	İşlevi	Kritik prop / olay
`View`	Yerleşim kapsayıcı	`style`, `flexDirection`
`Text`	Metin gösterir	`numberOfLines`, `style`
`Image`	Yerel veya uzak görsel	`source`, `resizeMode`, boyut
`TextInput`	Kullanıcıdan metin	`value`, `onChangeText`, `keyboardType`
`Button`	Basit buton	`title`, `onPress`, `disabled`
`TouchableOpacity`	Özelleştirilebilir dokunuş	`onPress`, `activeOpacity`
`ScrollView`	Tüm çocukları birlikte render	`contentContainerStyle`
`FlatList`	Sanallaştırılmış liste	`data`, `renderItem`, `keyExtractor`
`SafeAreaView`	Çentik/sistem alanı uyumu	Projede kullanılan kütüphane
`Picker`	Açılır seçim	`selectedValue`, `onValueChange`
`RadioButton`	Tek seçenek seçimi	`value`, `status`, `onPress` (pakete göre)
`Checkbox`	Aç/kapat veya çoklu seçim	`value`, `onValueChange` (pakete göre)

2. View, Text, Image

```
import { View, Text, Image } from 'react-native';
export default function Profil() {
  return <View style={{ padding: 20, alignItems: 'center' }}>
    <Image source={{ uri: 'https://picsum.photos/120' }}
           style={{ width: 120, height: 120, borderRadius: 60 }} />
    <Text style={{ fontSize: 20, fontWeight: 'bold' }}>Öğrenci</Text>
  </View>;
}
```

Tuzak: Uzaktan gelen `Image` çoğu durumda açık `width` / `height` gerektirir. RN'de metni doğrudan `View` içine değil, `Text` içine yaz. `source={require('./assets/logo.png')}` statik yerel görsel örneğidir.

3. TextInput, Button, TouchableOpacity

```
import { useState } from 'react';
import { View, Text, TextInput, TouchableOpacity } from 'react-native';
export default function Giris() {
  const [eposta, setEposta] = useState('');
  const gecerli = /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(eposta);
  return <View style={{ padding: 20, gap: 12 }}>
    <TextInput placeholder="E-posta" value={eposta}
      onChangeText={setEposta} keyboardType="email-address"
      autoCapitalize="none" style={{ borderWidth: 1, padding: 10 }} />
    <TouchableOpacity disabled={!gecerli} onPress={() => console.log(eposta)}
      style={{ padding: 14, backgroundColor: gecerli ? '#2563eb' : '#94a3b8' }}>
      <Text style={{ color: '#fff' }}>Gönder</Text>
    </TouchableOpacity>
  </View>;
}
```

`TouchableOpacity` içindeki içeriğin tasarımını kendin düzenlersin; `Button` daha basittir. Güncel kodda `Pressable` da tercih edilebilir. `secureTextEntry` parola girişinde kullanılır, ancak tek başına güvenli kimlik doğrulama sağlamaz.

4. ScrollView ve FlatList farkı

```
import { FlatList, Text } from 'react-native';
const veri = [{ id: '1', ad: 'İlk' }, { id: '2', ad: 'İkinci' }];
export default function Liste() {
  return <FlatList data={veri} keyExtractor={x => x.id}
    renderItem={({ item }) => <Text style={{ padding: 12 }}>{item.ad}</Text>}
    ListEmptyComponent={<Text>Veri yok</Text>} />;
}
```

Durum	Tercih	Neden
Kısa statik form / birkaç kutu	`ScrollView`	Basit yerleşim
Uzun / sunucudan gelen liste	`FlatList`	Sanallaştırma, performans
Liste başlığı	`ListHeaderComponent`	Listeyle birlikte kaydır
Aşağı çekince yenile	`refreshing` , `onRefresh`	Yeniden yükleme
Sayfalama	`onEndReached`	Daha fazla kayıt yükle

Tuzak: `ScrollView` tüm çocukları oluşturur; binlerce kayıt için gereksiz bellek yükü. `FlatList` 'i rastgele `ScrollView` içine aynı yönde yerleştirmek sanallaştırmayı bozabilir.

5. Seçim bileşenleri: kurulum ve desen

React Native çekirdeğinde `Picker` , `Checkbox` , `RadioButton` isimli tek bir standart bileşen seti yoktur; ders notları paket tabanlı örnekler verir. Örneğin Expo projesinde:

```
npx expo install @react-native-picker/picker
npx expo install expo-checkbox
# RadioButton için seçilen UI kitaplığının kurulumunu ayrıca kontrol et.
```

```
import { Picker } from '@react-native-picker/picker';
import Checkbox from 'expo-checkbox';
// state: const [sehir, setSehir] = useState('ist');
// state: const [kabul, setKabul] = useState(false);
<Picker selectedValue={sehir} onValueChange={setSehir}>
  <Picker.Item label="İstanbul" value="ist" />
  <Picker.Item label="Ankara" value="ank" />
</Picker>
<Checkbox value={kabul} onValueChange={setKabul} />
```

Not: Yukarıdaki iki JSX satırı bir bileşenin `return` bölümüne yerleştirilmeli; bağımsız tam `App` değildir. RadioButton tek seçim, checkbox bağımsız aç/kapa veya çoklu seçim için uygundur.

6. Safe area ve etkileşim tuzakları

`SafeAreaView` için kullandığın React Native / Expo sürümüne göre çekirdek sürüm yerine `react-native-safe-area-context` önerilebilir. Bir ekranda hem klavye hem çentik hem kaydırma bulunuyorsa yalnız `paddingTop` sabit sayısını kullanmak tüm cihazlarda doğru değildir.

S: `renderItem` ne alır? Genelde `{ item, index }` içeren parametre; tek öğeyi JSX'e çevirir.

S: `keyExtractor` neden? Stabil liste kimlikleri sağlamak için.

S: `Image` neden görünmüyor? `source` , URI, izin/ağ ve görünür boyut kontrol edilir.

S: `onChangeText` mi `onChange` mi? Basit string state için genelde `onChangeText` .

Kaynak: https://github.com/zeki-ozen/MobileAppDevelopment/tree/main/W6

7. HAFTA | Veri Saklama ve Yerel Veritabanı

> **Kaynak:** `W7/README.md` , `AsyncStorage.md` , `SQLite.md` , `Realm.md` , `MMKV.md` , `MySQL.md` . Depodaki örneklerde Expo istemci + Express REST API akışı kullanılır.

30 saniyelik hızlı bakış

State kalıcı değildir: Uygulama kapanınca sıradan `useState` kaybolur. **AsyncStorage** asenkron key-value; **MMKV** hızlı key-value; **SQLite** yerel ilişkisel tablolar/SQL; **Realm** nesne tabanlı yerel veritabanı; **MySQL** genellikle **sunucuda** çalışır, mobil uygulama veriye doğrudan veritabanı parolasıyla bağlanmaz; arada API bulunur.

1. Hangi saklama yöntemini seçmeliyim?

Seçenek	Veri modeli	Uygun olduğu iş
`useState`	Bellek içi	Geçici ekran durumu
`AsyncStorage`	Asenkron string key-value	Basit tercihler, küçük cache
`MMKV`	Hızlı key-value	Performans odaklı yerel ayarlar
`SQLite`	Tablo, satır, SQL	İlişkili/yapılandırılmış çevrimdışı veriler
`Realm`	Nesne tabanlı veritabanı	Yerel nesne modeli ve sorgular
`MySQL + REST`	Sunucudaki ilişkisel DB	Merkezi veri ve çoklu cihaz senkronizasyonu

Güvenlik: AsyncStorage şifreli güvenli kasa değildir. Parolayı/erişim tokenını doğrudan düz metin saklama; hassas sırlar için platformun güvenli saklama API'lerini (`expo-secure-store` vb.) değerlendir.

2. AsyncStorage: tam kaydet/oku/sil kalıbı

```
npx expo install @react-native-async-storage/async-storage
```

```
import AsyncStorage from '@react-native-async-storage/async-storage';
const KEY = 'kullanici_ayar';

export async function ayarKaydet(ayar) {
  await AsyncStorage.setItem(KEY, JSON.stringify(ayar));
}
export async function ayarOku() {
  const json = await AsyncStorage.getItem(KEY);
  return json === null ? null : JSON.parse(json);
}
export async function ayarSil() {
  await AsyncStorage.removeItem(KEY);
}
```

Neden `JSON.stringify` ? AsyncStorage string saklar; nesne kaydederken JSON string'e dönüştürülür; okurken `JSON.parse` ile nesneye çevrilir. Bozuk JSON ihtimali varsa parse işlemini `try/catch` içine al.

3. SQLite: tablo ve CRUD

`expo-sqlite` API'si SDK sürümüne göre değişebilir. Aşağıdaki örnek güncel `openDatabaseAsync` / `runAsync` desenini gösterir; depodaki bazı örnekler eski `openDatabase` / `transaction` biçimindedir. **Proje SDK'sının dokümantasyonunu temel al.**

```
npx expo install expo-sqlite
```

```
import * as SQLite from 'expo-sqlite';
export async function ornekVeritabaniIslemi() {
  const db = await SQLite.openDatabaseAsync('notlar.db');
  await db.execAsync(`CREATE TABLE IF NOT EXISTS notlar (
    id INTEGER PRIMARY KEY NOT NULL, baslik TEXT NOT NULL
  );`);
  const sonuc = await db.runAsync('INSERT INTO notlar (baslik) VALUES (?)', ['Sınav']);
  const liste = await db.getAllAsync('SELECT * FROM notlar ORDER BY id DESC');
  await db.runAsync('UPDATE notlar SET baslik=? WHERE id=?', ['Vize', sonuc.lastInsertRowId]);
  await db.runAsync('DELETE FROM notlar WHERE id=?', [sonuc.lastInsertRowId]);
  return liste;
}
```

SQL dikkat: Değerleri `?` parametreleriyle geçir. Kullanıcı girdisini SQL metnine doğrudan birleştirme; SQL injection riski doğar.

4. MMKV ve Realm

MMKV: Depo örnekleri `new MMKV()` ve `storage.getString('settings')` / `storage.set('settings', value)` mantığını anlatır. API sürümü ve Expo Go uyumluluğu için paket dokümanını incele; bazı native modüller özel development build gerektirebilir.

```
// Kavramsal MMKV kullanım kalıbı (sürüm / entegrasyona göre değişebilir):
const settings = { theme: 'dark' };
storage.set('settings', JSON.stringify(settings));
const result = JSON.parse(storage.getString('settings') ?? '{}');
```

Realm: Nesne şeması tanımlama, transaction içinde yazma, sorgulama gibi veritabanı işlemleri sunar. Mobil uygulamada kalıcı veri gereksinimi, paket durumu ve sürüm desteği değerlendirilmelidir.

5. MySQL neden istemcide direkt kullanılmamalı?

```
Mobil uygulama → HTTPS / REST API → Express sunucu → MySQL
Mobil uygulama ← JSON yanıt      ← İşlenmiş veri ← SQL sonucu
```

Veritabanı kullanıcı adı/parolası mobil paketin içine gömülürse uygulamayı edinen kişiler tarafından çıkarılabilir. Yetkilendirme, sorgu doğrulama ve veriye erişim sunucu tarafında yapılmalı. React Native uygulaması çoğunlukla `fetch` / `axios` ile API'yi çağırır.

6. Çevrimdışı çalışma + senkronizasyon

1. Önce yerel önbelleği oku, ekranı bekletmeden göster.
2. İnternet varsa API'den taze veri al.
3. Gelen veriyi doğrula, yerel kayda güncelle.
4. Çevrimdışı yapılan değişiklikleri kuyrukla.
5. Tekrar çevrimiçi olduğunda gönder; tekrar eden istek ve çakışma politikası planla.

Tuzak: AsyncStorage `getItem` bir `Promise` döndürür. `const x = AsyncStorage.getItem('k')` doğrudan veri değil Promise'tir; `await` gerekir.

7. Sınav mini soru-cevap

S: `useState` ile AsyncStorage farkı? State geçicidir, AsyncStorage süreçler arasında veri saklar.

S: SQLite ile AsyncStorage farkı? SQLite tablo/SQL/sorgu; AsyncStorage basit key-value.

S: MySQL'e doğrudan telefondan bağlanılır mı? Güvenlik/mimari gereği genellikle hayır; API katmanı kullanılır.

S: `JSON.parse` neden hataya düşebilir? Kayıt geçerli JSON değilse; `try/catch` kullan.

Kaynak: https://github.com/zeki-ozen/MobileAppDevelopment/tree/main/W7

8. HAFTA | API, Fetch, Axios ve Asenkron Veri

> **Kaynak:** `W8/README.md` , `FetchAPI.md` , `Axios.md` , `AsyncAwait.md` , `ErrorHandling.md` , `RESTfulAPIs.md` , `LoadingStates.md` .

30 saniyelik hızlı bakış

HTTP istek → yanıt → JSON → state → render. `fetch` yerleşik istemci; Axios haricî kütüphane. `async/await` Promise tabanlı işlemleri okunaklı yazar. `fetch` **4xx/5xx için kendiliğinden reject etmez**: `response.ok` kontrol et. Her ekran için **loading / error / success / empty** durumlarını düşün.

1. REST ve HTTP tablo

Metot	Amaç	Sık karşılaşılan yanıt
`GET /items`	Liste oku	`200 OK`
`GET /items/5`	Tek kayıt oku	`200` , `404`
`POST /items`	Yeni kayıt oluştur	`201 Created`
`PUT /items/5`	Kaynağı değiştir	`200` , `204`
`PATCH /items/5`	Kısmi güncelle	`200` , `204`
`DELETE /items/5`	Sil	`200` , `204`

Kod aileleri: `2xx` başarılı; `4xx` istemci/istek/erişim; `5xx` sunucu. JSON gönderirken genelde `Content-Type: application/json` ve `JSON.stringify` kullanılır.

2. Fetch ile GET: güvenli temel desen

```
async function kullanicilariGetir() {
  const response = await fetch('https://jsonplaceholder.typicode.com/users');
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  return await response.json();
}

async function ornekKullanim() {
  try {
    const liste = await kullanicilariGetir();
    console.log(liste.length);
  } catch (e) {
    console.error('İstek başarısız:', e.message);
  }
}
ornekKullanim();
```

Sırayla: `fetch` Promise döndürür → `await` yanıtı bekler → `ok` 200–299 kontrolüdür → `json()` gövdeyi ayrıştırır → hata `catch` içinde yakalanır.

3. Çalışan RN örneği: loading/error/empty/data

```
import { useEffect, useState } from 'react';
import { View, Text, Button, FlatList, ActivityIndicator } from 'react-native';

const URL = 'https://jsonplaceholder.typicode.com/users';
export default function App() {
  const [veriler, setVeriler] = useState([]);
  const [yukleniyor, setYukleniyor] = useState(true);
  const [hata, setHata] = useState('');
  const [yenile, setYenile] = useState(0);

  useEffect(() => {
    const controller = new AbortController();
    async function yukle() {
      setYukleniyor(true);
      setHata('');
      try {
        const response = await fetch(URL, { signal: controller.signal });
        if (!response.ok) throw new Error(`HTTP ${response.status}`);
        const data = await response.json();
        setVeriler(data);
      } catch (e) {
        if (e.name !== 'AbortError') setHata(e.message);
      } finally {
        if (!controller.signal.aborted) setYukleniyor(false);
      }
    }
    yukle();
    return () => controller.abort();
  }, [yenile]);

  if (yukleniyor) return <ActivityIndicator style={{ flex: 1 }} />;
  if (hata) return <View style={{ padding: 20 }}>
    <Text>Hata: {hata}</Text><Button title="Tekrar dene" onPress={() => setYenile(n => n + 1)} />
  </View>;
  return <FlatList data={veriler} keyExtractor={x => String(x.id)}
    ListEmptyComponent={<Text>Kayıt bulunamadı.</Text>}
    renderItem={({ item }) => <Text style={{ padding: 12 }}>{item.name}</Text>} />;
}
```

Neden `AbortController`? Bileşen ayrıldığında eski isteği iptal ederek gereksiz güncelleme ve yarış koşullarını azaltmak için. Bazı API'lerde platform/sürüm uyumluluğu kontrol edilir.

4. POST: JSON oluşturma isteği

```
async function yeniKayitOlustur(baslik) {
  const res = await fetch('https://jsonplaceholder.typicode.com/posts', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ title: baslik, body: 'Metin', userId: 1 }),
  });
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return res.json();
}
```

JSONPlaceholder gibi örnek servisler yazma isteğine örnek yanıt verebilir, fakat kalıcı gerçek sunucu verisi yaratmayabilir. **`GET` için body kullanma; URL parametresi / query kullan.**

5. Axios karşılaştırması

```
npm install axios
```

```
import axios from 'axios';
const cevap = await axios.get('https://jsonplaceholder.typicode.com/users');
console.log(cevap.data); // Axios veri alanı

await axios.post('https://example.com/items', { title: 'Not' });
```

Özellik	Fetch	Axios
Ek kurulum	Genelde gerekmez	`axios` paketi
JSON yanıta erişim	`await res.json()`	`response.data`
4xx / 5xx	`response.ok` kontrolü	Varsayılan olarak reject
İstek/yanıt interceptor	Yerleşik değil	Var

Özellik	Fetch	Axios
İptal	`AbortController`	`signal` desteği

6. Async/await ve Promise hataları

```
// Ardışık: ikinci işlem ilkine bağımlıysa
const user = await fetchUser();
const posts = await fetchPosts(user.id);

// Paralel: iki istek birbirinden bağımsızsa
const [a, b] = await Promise.all([fetchA(), fetchB()]);
```

Tuzak: `array.forEach(async item => ...)` tamamlanmasını beklemez. Toplu paralel iş için `await Promise.all(array.map(async item => ...))`; sırayla yapmak için `for...of` + `await` kullan.

7. Ağ hata türü ve kullanıcı deneyimi

Durum	Görünüm/işlem
Yükleniyor	`ActivityIndicator`, boş spinner değil açıklama
Ağ hatası	Açık mesaj + tekrar dene
HTTP 401/403	Oturum/izin kontrolü
HTTP 404	Kayıt bulunamadı
Boş başarı yanıtı	“Henüz kayıt yok”
Başarılı dolu veri	Liste veya detay

Mobil cihaz notu: Gerçek telefonda `localhost`, telefonun kendisini ifade eder; bilgisayarda çalışan Express sunucusuna bağlanmak için uygun yerel ağ IP'si veya erişilebilir bir sunucu adresi gerekir.

8. Sınav mini soru-cevap

S: Fetch'te `catch` neden 404'ü yakalamıyor? Çünkü `fetch` HTTP hata durumunda çoğunlukla başarılı şekilde Response döndürür; `!res.ok` kontrolü gerekir.

S: `finally` ne için? Hata ya da başarıdan bağımsız temizleme/loading kapatma işlemleri.

S: Axios'ta veri nerede? `response.data`.

S: `async` fonksiyon ne döndürür? Daima Promise.

Kaynak: https://github.com/zeki-ozen/MobileAppDevelopment/tree/main/W8

9. HAFTA | Kamera, Bildirim ve Haritalar

> **Kaynak:** `W9/README.md` , `components/Camera.md` , `Notifications.md` , `ReactNativeMaps.md` , `ReactNativeLeaflet.md` .

30 saniyelik hızlı bakış

Kamera: `expo-camera` , `CameraView` , `useCameraPermissions` , `takePictureAsync` , `onBarcodeScanned` . **Bildirim:** `expo-notifications` , kullanıcı izni ve yerel bildirim zamanlama. **Harita:** `react-native-maps` → `MapView` , `Marker` , `Polyline` , `region` ; Leaflet alternatifi → `WebView` içinde HTML harita. **Cihaz izni reddedilirse** alternatif arayüz göster.

1. Kütüphaneler ve izin mantığı

Özellik	Paket / bileşen	Önemli nokta
Fotoğraf çekme	`expo-camera` / `CameraView`	Kamera izni, ref
QR / barkod	`CameraView` / `onBarcodeScanned`	Aynı kodun tekrar okunmasını durdur
Yerel bildirim	`expo-notifications`	Bildirim izni ve handler
Harita	`react-native-maps`	Bölge ve koordinat
Marker	`Marker`	`coordinate={{ latitude, longitude }}`
Rota çizgisi	`Polyline`	Sıralı koordinat dizisi
Leaflet	`react-native-webview`	HTML/JS harita, ağ erişimi

```
npx expo install expo-camera expo-notifications react-native-maps
# Leaflet tabanlı alternatif kullanılacaksa:
npx expo install react-native-webview
```

Sürüm uyarısı: Expo SDK, işletim sistemi ve development build / Expo Go desteği zaman içinde değişir. Paketleri `npx expo install` ile SDK ile eşleştir. Gerçek cihaz izinleri ve push bildirimleri için ayrıca platform ayarı/test gerekebilir.

2. Kamera: izin ver, çek, göster

```
import { useRef, useState } from 'react';
import { Button, Image, Text, View } from 'react-native';
import { CameraView, useCameraPermissions } from 'expo-camera';

export default function KameraEkrani() {
  const kamera = useRef(null);
  const [izin, izinIste] = useCameraPermissions();
  const [resim, setResim] = useState(null);
  if (!izin) return <Text>İzin kontrol ediliyor...</Text>;
  if (!izin.granted) return <View>
    <Text>Kamera izni gerekli.</Text>
    <Button title="İzin ver" onPress={izinIste} />
  </View>;
  return <View style={{ flex: 1 }}>
    <CameraView ref={kamera} style={{ flex: 1 }} facing="back" />
    <Button title="Fotoğraf çek" onPress={async () => {
      const foto = await kamera.current?.takePictureAsync();
      if (foto?.uri) setResim(foto.uri);
    }} />
    {resim && <Image source={{ uri: resim }} style={{ width: 140, height: 140 }} />}
  </View>;
}
```

Akış: Hook izin durumunu verir → izin yoksa izin ekranı → kamera görüntüsü → `ref.current.takePictureAsync()` → dönen dosya URI'si ile `Image` . Hata/cihaz emülatörü davranışları için gerçek cihaz testi gerekir.

3. Barkod / QR kodu

```
// CameraView içine eklenen temel desen:
<CameraView style={{ flex: 1 }}
  barcodeScannerSettings={{ barcodeTypes: ['qr'] }}
  onBarcodeScanned={({ data }) => {
    if (!taramaAcik) return;
    setTaramaAcik(false);  // birden fazla tetiklemeyi önle
    setSonKod(data);
  }}
/>
```

`taramaAcik` ve `setSonKod` bu kodu kullanan üst bileşende `useState` ile tanımlanmalıdır. Barkod verisini URL diye **doğrulamadan açma**; kullanıcı girdisi gibi değerlendir.

4. Yerel bildirim: izin ve zamanlama

```
import * as Notifications from 'expo-notifications';

async function bildirimGonder() {
  const mevcut = await Notifications.getPermissionsAsync();
  let durum = mevcut.status;
  if (durum !== 'granted') {
    const istek = await Notifications.requestPermissionsAsync();
    durum = istek.status;
  }
  if (durum !== 'granted') return false;

  await Notifications.scheduleNotificationAsync({
    content: { title: 'Hatırlatma', body: 'Çalışma zamanı!' },
    trigger: null, // yerel, hemen planla
  });
  return true;
}
```

Not: Bu, **yerel bildirim** örneğidir. Sunucudan cihazlara push göndermek için token, izin, platform kimliği ve servis yapılandırması gibi ilave adımlar vardır. Bildirimin ön planda görünme davranışı için `setNotificationHandler` yapılandırması/SDK sürümü önemlidir.

5. Harita: MapView + Marker

```
import { View } from 'react-native';
import MapView, { Marker } from 'react-native-maps';
const merkez = {
  latitude: 41.015137, longitude: 28.97953,
  latitudeDelta: 0.05, longitudeDelta: 0.05,
};
export default function Harita() {
  return <View style={{ flex: 1 }}>
    <MapView style={{ flex: 1 }} initialRegion={merkez}>
      <Marker coordinate={{ latitude: 41.015137, longitude: 28.97953 }}
              title="İstanbul" description="Örnek işaretçi" />
    </MapView>
  </View>;
}
```

- `latitude`, `longitude`: Konum koordinatları.
- `latitudeDelta`, `longitudeDelta`: Yakınlaştırma/görünüm kapsamı.
- `initialRegion`: Başlangıç konumu; `region` kontrollü harita bölgesi.
- `onRegionChangeComplete`: Kullanıcı haritayı kaydırınca yeni bölgeyi takip et.
- `Polyline`: Koordinat dizisinden çizgi üretir; bu tek başına yol tarifi hesaplamak **değildir**.

6. Leaflet / WebView alternatifinin mantığı

```
import { WebView } from 'react-native-webview';
const html = `<!DOCTYPE html><html><body>
  <h3>Harita gösterim alanı</h3>
  <div id="map" style="height:300px"></div>
  <!-- Leaflet CSS/JS, harita döşemesi ve init kodu eklenir -->
</body></html>`;
export default function HaritaWeb() {
  return <WebView originWhitelist={['*']} source={{ html }} />;
}
```

Bu **sadece WebView yerleştirme iskeletidir**, çalışır Leaflet haritası için HTML'ye Leaflet kütüphanesi, tile provider ve `L.map(...)` kurulumu eklenmelidir. `react-native-maps` native yaklaşım; Leaflet ise web içeriği çalıştırır.

7. Yaygın hata / teşhis

Belirti	Kontrol et
Siyah kamera ekranı	İzin, gerçek cihaz, emülatör, SDK desteği
Fotoğraf alınmıyor	`ref` , `takePictureAsync` , kamera hazır mı?
Bildirim görünmüyor	İzin, foreground handler, OS ayarı
Harita boş	Ağ, API anahtarı/platform ayarları, harita stili
Marker görünmüyor	Koordinatların numeric olması, harita bölgesi
QR sürekli tetikleniyor	Taramadan sonra handler'ı devre dışı bırak

8. Sınav mini soru-cevap

S: Kamera neden runtime izin gerektirir? Donanıma/kişisel veriye erişim kullanıcı onayına bağlıdır.

S: `initialRegion` ve `region` ? İlki başlangıç alanı; ikincisi state tarafından sürekli kontrol edilen alan.

S: Yerel bildirim vs push? Yerel bildirim cihazda planlanır; push uzaktaki sunucu/servis üzerinden iletilir.

S: WebView içinde Leaflet ile `react-native-maps` farkı? İlki web teknolojilerini bir kapsayıcıda; ikincisi native harita bileşeni kullanır.

Kaynak: https://github.com/zeki-ozen/MobileAppDevelopment/tree/main/W9